

Habib University
Dhanani School of Science and Engineering
Computer Architecture
EE 371 / CS 330 / CE 321 (Spring 2026)
Homework 3 Solution

Release date: 27th March 2026 **Due date:**
Total marks: 100 **Obtained Marks:**
Student Name: **Student ID:**

Instructions:

1. This assignment should be done in pairs.
2. All questions should be answered in **black ink only**.
3. Scan your answer sheet and upload it on HU LMS before the due date.
4. Take a print of the assignment and solve (provide answers) in the space provided after every question. You can use extra sheets for your answers. Attach them properly.
5. You can also be asked to give a viva where you will be judged whether you understood the question yourself or not. If you are unable to correctly answer the question you have attempted right, you may lose your marks.
6. Zero will be given if the assignment is found to be plagiarized.
7. **No assignment shall be graded if submitted late.**

CLO Assessment:

This assignment assesses students for the following course learning outcomes.

Course Learning Outcomes		CLO Assessed
CLO 1	<i>Explain</i> the role of ISA in modern processors and instruction encodings and assembly language programming	
CLO 2	<i>Explain</i> the architecture and working of a single cycle processor	
CLO 3	<i>Design</i> the architecture to mitigate issues of a pipelined processor	✓
CLO 4	<i>Analyze</i> the performance of cache operations	

Question # 1 (Points: 12)

A 5-stage pipelined processor has the following stage latencies:

IF	ID	EX	MEM	WB
220ps	200ps	320ps	260ps	180ps

and also consider the inter-stage register delay to be 25ps.

Also, assume that instructions executed by the processor for a program-A are divided as follows:

ALU/Logic	Jump/Branch	Load	Store
40%	10%	25%	25%

1. Compute the clock cycle time for both pipelined and non-pipelined versions. (2 points)

Solution:

Non-pipelined processor

$$\begin{aligned}
 T_{\text{non-pipe}} &= \text{IF} + \text{ID} + \text{EX} + \text{MEM} + \text{WB} \\
 &= 220 + 200 + 320 + 260 + 180 \\
 &= 1180 \text{ ps}
 \end{aligned}$$

So each instruction takes **1180 ps** in the non-pipelined design.

Pipelined processor

In a pipeline, the clock period must accommodate the longest stage plus the inter-stage register delay:

$$\begin{aligned}
 T_{\text{pipe}} &= \max(\text{IF}, \text{ID}, \text{EX}, \text{MEM}, \text{WB}) + t_{\text{reg}} \\
 &= \max(220, 200, 320, 260, 180) + 25 \\
 &= 320 + 25 = 345 \text{ ps}
 \end{aligned}$$

So pipeline clock period is **345 ps**.

2. Determine the total latency for a store instruction in both processors. (2 points)

Solution:

Non-pipelined store latency

Since in a single-cycle design latency will be determined by the clock cycle time therefore, store's latency is:

$$\text{Store}_{\text{non-pipe}} = 1180 \text{ ps} = 1.18 \text{ ns}$$

So non-pipelined store latency is **1180 ps**.

Pipelined store latency

As latency is from the instruction start till end and again for pipeline design the latency will be determined through the cycle time as all instructions have to go through the 5 pipeline stages therefore, store's latency is:

$$\text{Store}_{\text{pipe}} = 5 \times T_{\text{pipe}} = 5 \times 345 = 1725 \text{ ps} = 1.725 \text{ ns}$$

So pipelined store latency is **1725 ps**.

3. If 1200 instructions are executed with no hazards, calculate the speedup achieved. (2 points)

Solution:

We compute execution time for both designs and then their ratio.

Non-pipelined execution time

Each instruction takes $T_{\text{non-pipe}} = 1180 \text{ ps}$, so:

$$T_{\text{non}} = 1200 \times 1180 = 1,416,000 \text{ ps} = 1.416 \text{ } \mu\text{s}$$

Pipelined execution time (no hazards)

For a pipeline with k stages, the total number of cycles to complete N instructions (steady pipeline, no hazards) is:

$$\text{cycles} = N + (k - 1)$$

Here $k = 5$. Thus $\text{cycles} = 1200 + (5 - 1) = 1204$ cycles. Each cycle takes $T_{\text{pipe}} = 345 \text{ ps}$, hence:

$$T_{\text{pipe,total}} = 1204 \times 345 = 415,380 \text{ ps} = 415.38 \text{ ns}$$

Speedup

$$\text{Speedup} = \frac{T_{\text{non}}}{T_{\text{pipe,total}}} = \frac{1,416,000}{415,380} \approx 3.409 \approx \mathbf{3.41}$$

4. Now assume a stall occurs every five instructions. What is the new speedup? (2 points)

Solution:

A stall every five instructions means we insert one bubble (one extra cycle) per 5 instructions.

Number of stalls for 1200 instructions

$$\text{stalls} = \frac{1200}{5} = 240.$$

(Assume this divides evenly.)

New pipeline cycle count

Previously, without stalls, cycles = 1204. Adding 240 stall cycles:

$$\text{cycles}_{\text{stalled}} = 1204 + 240 = 1444.$$

Execution time with stalls

$$T_{\text{stalled}} = 1444 \times 345 = 498,180 \text{ ps.}$$

New speedup

$$\text{Speedup}_{\text{stalled}} = \frac{1,416,000}{498,180} \approx 2.842 \approx \mathbf{2.84}.$$

5. Suggest a possible architectural modification to reduce overall cycle time without changing ISA behavior. Calculate the new pipeline clock cycle time. (2 points)

Solution:

Suggested modification

Split the longest stage (EX, 320 ps) into two smaller stages (say EX1 and EX2) so that no single stage is longer than the next-largest stage (MEM = 260 ps). This does not change ISA semantics; it only adds another pipeline register and splits EX into two pipeline stages.

Re-evaluate stage latencies after splitting EX

If EX is split into two equal parts:

$$EX1 \approx 160 \text{ ps}, \quad EX2 \approx 160 \text{ ps}.$$

The stage latencies become:

$$220, 200, 160, 160, 260, 180.$$

The new maximum stage latency is **260 ps (MEM)**.

New pipeline clock period

Add the register overhead:

$$T_{\text{new-pipe}} = 260 + 25 = 285 \text{ ps}.$$

So with this modification, the pipeline clock reduces from **345 ps to 285 ps**.

6. Assuming all stages are fully utilized, calculate the utilization of the instruction memory and data memory over 1000 instruction executions. (2 points)

Solution:

Instruction memory utilization

Every instruction fetch uses instruction memory once (in IF). Thus, during 1000 instruction executions:

$$\text{Instruction memory accesses} = 1000.$$

Instruction memory utilization = **100%**.

Data memory utilization

Data memory is accessed by loads and stores only. From the instruction mix:

$$\text{Loads} = 25\%, \quad \text{Stores} = 25\% \Rightarrow 50\% \text{ access data memory}.$$

For 1000 instructions:

$$\text{Data memory accesses} = 0.50 \times 1000 = 500.$$

Thus data memory utilization = **50%**.

Question # 2 (Points: 10)

In the RISC-V single-cycle processor design, load (ld) is the instruction with the longest latency. If we modify the memory instructions so that there is no address calculation (i.e., the address to be loaded from/stored to must be calculated and placed in rs1 before calling ld/sd), then no instruction would use both the ALU and Data memory. This would allow us to reduce the clock cycle time of the single-cycle implementation. However, it would also increase the number of instructions, because many ld and sd instructions would need to be replaced with ld/add or sd/add combinations.

$$\begin{aligned} \text{ld } x18, 100(x5) &\Rightarrow \text{addi } x5, x0, 100 \\ &\quad \text{ld } x18, x5 \end{aligned}$$

1. For the latencies as mentioned in Question 1, what would the new clock cycle time be for single-cycle (non-pipelined) processor? (1 mark)

Solution:

The single cycle non-pipelined processor clock cycle time is determined by the longest-latency instruction. Since before the proposed modification, the longest instruction was ld. Now it no longer uses ALU and Memory both. Hence the latency can be determined by the maximum instruction latency:

New latency for different instructions

Ld = $220 + 200 + 260 + 180 = 860$ ps

Add = $220 + 200 + 320 + 180 = 920$ ps

Sd = $220 + 200 + 260 = 680$ ps

Beq = $220 + 200 + 320 = 740$ ps

Maximum latency is therefore of Arithmetic instructions so new clock cycle time will be 920 ps.

2. In the non-pipelined version, would a program with the instruction mix presented in Question 1 run faster or slower on this new CPU? And by how much? (For simplicity, assume every ld and sd instruction is replaced with a sequence of two instructions.) (1 mark)

Solution:

$$T_{\text{execution}} = (\text{total instructions}) \times (\text{clock cycle time})$$

$$T_1 (\text{before modification}) = I \times 1180$$

Let n be fraction of load and store instructions in the program. Then the new instruction count after the modification will be $I + n \cdot I$, because we are going to add $n \cdot I$ more instructions.

$$T_2 (\text{after modification}) = I(1 + n) \times 920$$

The fraction of affected instructions (ld + sd) = $25\% + 25\% = 50\% = (n=0.5)$

$$T_2 (\text{after modification}) = I (1.5) \times 920$$

$$\text{Performance factor: } \frac{T_1}{T_2} = \frac{I \times 1180}{1.5 I \times 920} = \frac{1180}{1380} \approx 0.855$$

The new CPU achieves only 85.5% of the performance of the old CPU, it would be slower.

3. What is the primary factor that influences whether a program will run faster or slower on the new CPU? (1 mark)

Solution:

The primary factor that influences whether a program will run faster or slower on the new CPU is the number of memory instructions in the program. If a program contains many ld and sd instructions, the instruction count increases significantly, potentially slowing down execution.

4. Do you consider the original CPU a better overall design; or do you consider the new CPU a better overall design? (1 mark)

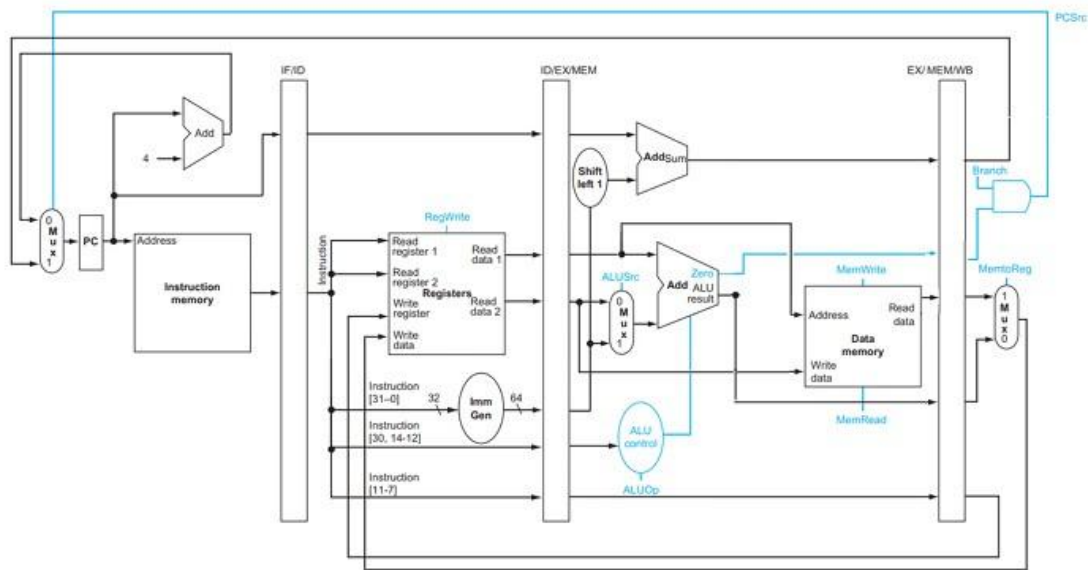
Solution:

- If the program has relatively many memory instructions, the original CPU is better since it reduces instruction count.
- If the program has relatively less memory instructions, the new CPU is better due to its faster clock cycle time.
- Using the above equation of part ii, the original CPU is likely a better design if the fraction of memory instructions is high (as seen in this case with 50%).

5. As a result of the change, the MEM and EX stages of the pipelined version of the processor can be overlapped and the pipeline has only four stages. Draw the modified pipelined datapath with control. Also mention the size of each inter-stage pipeline register. (3 marks)

Solution:

The datapath will have EX/MEM stage overlapped:



Size of each inter-stage pipeline register:

- IF/ID Register: Stores PC (64 bits), instruction (32 bits) → 96 bits
- ID/EX/MEM Register: Stores control signals (8 bits), read registers (64 bits each), immediate values (64 bits), PC (64 bits), rd (5 bits), funct3 and funct7 (10 bits) → ~279 bits
- EX/MEM/WB Register (merged stage): BTA (64 bits), ALU result (64 bits), memory result (64 bits), rd (5 bits), control signals (3 bits) → ~200 bits

(funct7 doesn't have to be 7 bits as only bit 30 is changing so we pass on only bit [12,13,14] of funct3 and bit [30] of funct7 so this register size could also be 271 bits)

6. How will the reduction in pipeline depth affect the cycle time? (1 mark)

Solution:

The cycle time would only change if the critical path changes but since memory operations are independent of execution unit both can work in parallel which means the cycle time is not going to change if it was dependent on either the EX stage or the MEM stage.

7. How might this change improve the performance of the pipeline? (1 mark)

Solution:

Running EX and MEM stages in parallel can effectively reduce the number of stalls as it will eliminate the need for a cycle or NOPs in case the program has a load instruction followed by an instruction that uses the resultant loaded register value.

8. How might this change degrade the performance of the pipeline? (1 mark)

Solution:

As discussed in previous parts, we are replacing the ld instruction with addi and ld and sd instruction with addi and sd. This will increase the number of instructions. And so the same program will require more instructions.

Secondly, we now expect the memory address to be directly read from the register file so there should be additional forwarding paths for the data memory in case a load/store instruction is dependent on an earlier operation, increasing its critical path.

e.g. add x5, x10, x11
 ld x6, x5

Question # 3 (Points: 10)

Consider the fragment of RISC-V assembly below:

```
label:  ld    x31, 32(x10)
        sd    x11, 8(x10)
        sub   x12, x14, x13
        add   x15, x12, x13
        beq   x24, x0, label
        add   x31, x31, x14
```

Suppose we modify the pipeline so that it has only one memory (that handles both instructions and data). In this case, there will be a structural hazard every time a program needs to fetch an instruction during the same cycle in which another instruction accesses data.

1. Draw a pipeline diagram to show where the above code will stall due to structural hazards. (3 marks)

Solution:

... indicates a stall/bubble

! indicates that the pipeline stage is inactive for that instruction

ld x31,32(x10)	IF	ID	EX	ME	WB							
sd x11,8(x10)		IF	ID	EX	ME	WB						
sub x12,x14,x13			IF	ID	EX	ME!	WB					
add x15,x12,x13				...	...	IF	ID	EX	ME!	WB		
beq x24,x0,label							IF	ID	EX	ME!	WB!	
add x31,x31, x14								IF	ID	EX	ME!	WB

2. In general, is it possible to reduce the number of stalls/NOPs resulting from this structural hazard by reordering code? (2 marks)

Solution:

In any case, there will be a structural hazard every time a program needs to fetch an instruction during the same cycle in which another instruction accesses data. Reordering the code will simply change the instructions in conflict. The number of stalls won't be reduced since every instruction has to be fetched. This means that data access for every instruction results in a stall.

3. We have seen that some data hazards can be eliminated by adding NOPs to the code. Can we do the same with this structural hazard? If so, explain how. If not, explain why not. (2 marks)

Solution:

Since NOP instructions also have to be fetched from instruction memory, they will also cause structural hazards. Therefore, we cannot solve a structural hazard using NOPs at the code level. So, it must be handled by the hardware.

4. Approximately how many stalls would you expect this structural hazard to generate in a typical program? Use the instruction mix from Question 1. (3 marks)

Solution:

Load and Store instructions use data memory. Hence,

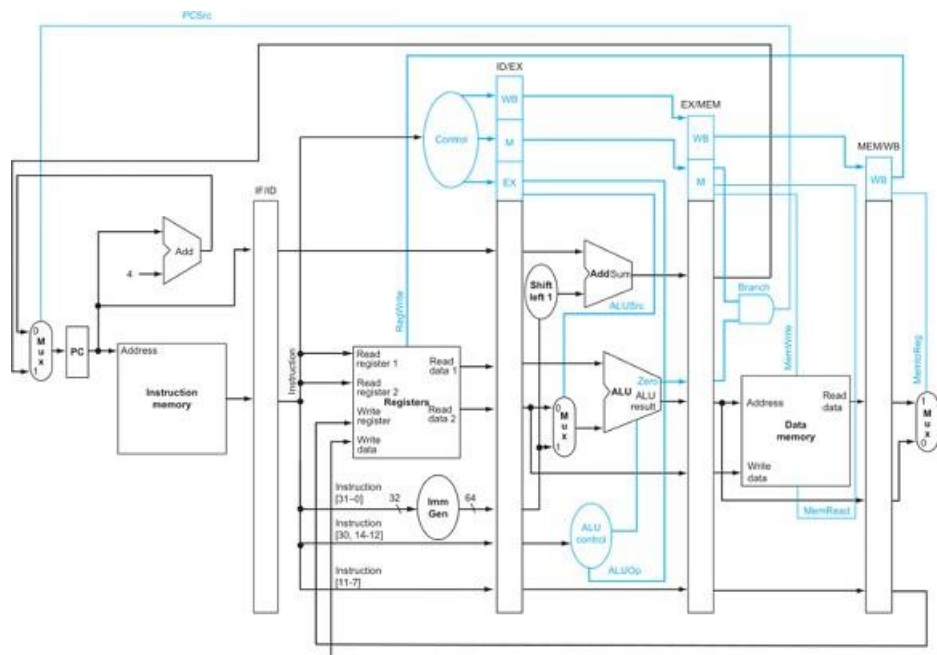
$$25\% + 25\% = 50\%$$

stalls are expected.

Question # 4 (Points: 15)

Suppose one of the following control signals in a single-cycle RISC-V processor is stuck-at-1 (always logic 1). Which instruction(s) would malfunction, what would be the issues observed and why?

Reference Figures:



1. RegWrite (1.8 marks)

Solution:

Instructions with no destination register would malfunction as RegWrite = 1 indicates that the ALU result has to be written in a destination register and those instructions do not have one.

These include S-Type, SB-Type and those instructions which do not write an rd.

2. MemRead (1.8 marks)

Solution:

MemRead is 1 only for load instructions. All the other instructions would malfunction if it gets stuck at 1 because they do not need to read data from memory.

These include R-type, S-Type, SB-type, I-type (except load), U-type, and UJ-type.

3. MemWrite (1.8 marks)

Solution:

MemWrite is 1 for instructions that need to write data to memory instead of registers. All the instructions that write to a destination register instead of memory would malfunction in this case.

These include R-Type, I-Type, SB-type, U-type, and UJ-type.

4. Branch (1.8 marks)

Solution:

If Branch is stuck at 1, it would make all the instructions as branch instructions even when they are not. The PC will use branch target instead of PC+4 for all the instructions which will result in incorrect fetching.

This would result in malfunction of all the instructions except branch instructions.

5. ALUSrc (1.8 marks)

Solution:

ALUSrc gives 1 for instructions whose second operand (readdata2) of ALU comes from immediate and not from rs2. If it is stuck at 1, all the instructions that use rs2 instead of immediate will malfunction.

These include R-type, S-type, SB-type.

6. PCSrc (1.8 marks)

Solution:

PCSrc selects between PC+4 and branch target address. If it is stuck at 1, it will always select branch target address to fetch next instruction, which will result in malfunctioning of branch not taken and all other instructions except branch taken.

7. ALUOp₁ (1.8 marks)

Solution:

ALUOp is a 2 bit signal that determines the operation ALU will be performing. It is mapped as 00 = add, 01 = sub, 10 = R-type. If ALUOp₁ is stuck at 1, load, store, and branch instructions will malfunction.

8. ALUOp₀ (1.8 marks)

Solution:

If ALUOp₀ is stuck at 1, R-type and load and store instructions which require add will malfunction.

Question # 5 (Points: 13)

The pipelined RISC-V processor executes the following code segment:

```
1. addi  x1, x0, 5      # x1 = 5
2. addi  x2, x0, 10     # x2 = 10
3. add   x3, x1, x2     # x3 = 15
4. sub   x4, x3, x1     # x4 = 10
5. lw    x5, 0(x4)      # load word from memory address in x4
6. add   x6, x5, x1     # x6 = x5 + x1
7. sw    x6, 4(x4)      # store word from x6 to memory[x4 + 4]
8. add   x7, x6, x3     # x7 = x6 + x3
```

Assume the following initial conditions:

- All registers are initialized to 0 except x0 (which is always 0).
- Memory location Mem[10] = 40.

1. Identify all data hazards present in the above sequence (specify the dependent registers and the type of hazard). (3 marks)

Solution:

- | | | |
|----------------------------------|-------------------------------|-----------------------|
| 1) Instr3 is dependent on Instr1 | (MEM-Hazard) | dependent register x1 |
| 2) Instr3 is dependent on Instr2 | (EX-Hazard) | dependent register x2 |
| 3) Instr4 is dependent on Instr3 | (EX-Hazard) | dependent register x3 |
| 4) Instr5 is dependent on Instr4 | (EX-Hazard) | dependent register x4 |
| 5) Instr6 is dependent on Instr5 | (Load-Use Hazard) | dependent register x5 |
| 6) Instr7 is dependent on Instr6 | (MEM/WB to Memory Forwarding) | dependent register x6 |
| 7) Instr8 is dependent on Instr7 | (MEM-Hazard) | dependent register x6 |

2. Assuming there is no forwarding and no stall detection, complete a table showing the value of each register (x1–x7) after each instruction finishes execution. Clearly indicate where incorrect values would appear due to hazards. (4 marks)

Solution:

X1	5	correct
X2	10	correct
X3	Cannot be determined with the information provided	incorrect
X4		incorrect
X5		incorrect
X6		incorrect
sw x6 4(x4)		incorrect
X7		incorrect

3. Describe how forwarding and hazard detection unit could be used to ensure correct execution of this program. Mention which hazards are resolved by forwarding and which require stalls. (3 marks)

Solution:

Referring to Q5 part1

- All the EX-Hazards can be resolved by forwarding from EX/MEM pipeline register to ALU (2, 3 and 4)
- All the MEM-Hazards can be resolved by forwarding from MEM/WB pipeline register to ALU (1, 7)
- Load-Use hazard will require one stall cycle and then can be forwarded from MEM/WB pipeline register to ALU
- Store instruction will need a different forwarding path from MEM/WB to Memory, not ALU and can also be done without stalls.

4. Redraw the pipeline diagram assuming full forwarding and hazard detection. Clearly label where stalls (if any) occur. (3 marks)

Solution:

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13
addi x1 x0 5	IF	ID	EX	MEM	WB								
addi x2 x0 10		IF	ID	EX	MEM	WB							
add x3 x1 x2			IF	ID	EX	MEM	WB						
sub x4 x3 x1				IF	ID	EX	MEM	WB					
lw x5 0(x4)					IF	ID	EX	MEM	WB				
NOP													
add x6 x5 x1							IF	ID	EX	MEM	WB		
sw x6 4(x4)								IF	ID	EX	MEM	WB	
add x7 x6 x3									IF	ID	EX	MEM	WB

Question # 6 (Points: 15)

Consider the following instruction that is not found in the RISC-V architecture:

Add to Memory

```
addm rd, Offset(rs)      // Reg[rd] = Reg[rd] +  
                          //      Mem[Offset + Reg[rs]]
```

We want to modify the RISC-V processor to support the above instruction. For parts (b) and (d) below, you can use a printed version of the figures in the book over which you can draw your suggested modifications (no need to draw the entire diagram from scratch).

1. Suggest if any of the existing instruction formats is a good choice to encode the new instruction. If not, then propose a new instruction format. (2 marks)

Solution:

I-type instruction format can be used, because it has an offset (immediate), a source and a destination register:

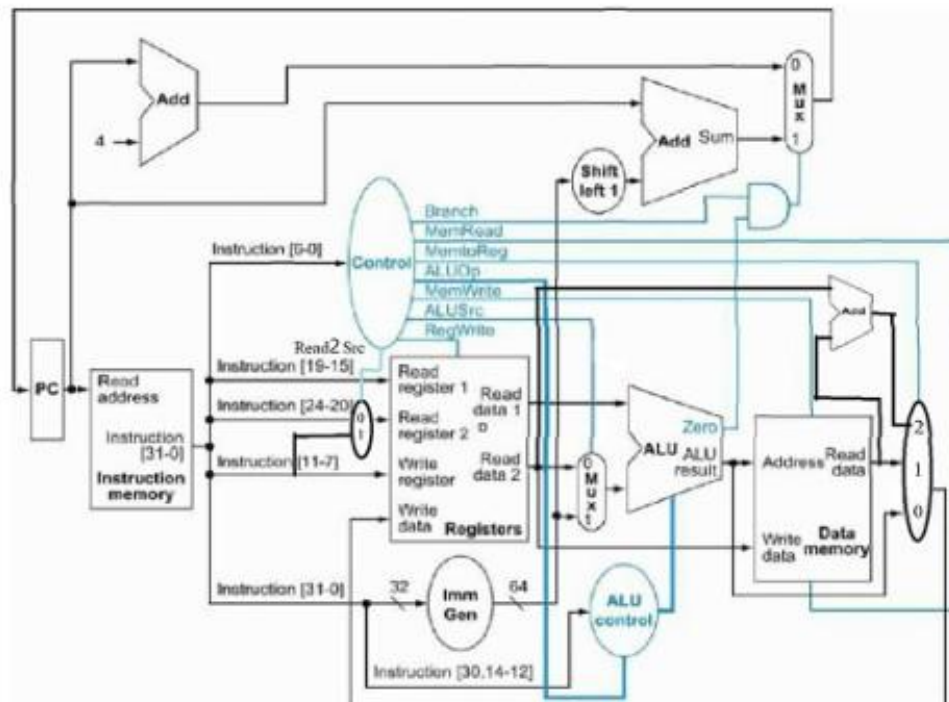
Immediate (12 bits) | rs1 (5 bits) | func3 (3 bits) | rd (5 bits) | opcode (7 bits)

2. Modify the datapath and control signals of the single-cycle RISC-V processor (Figure 4.17 of the book) to execute the new instruction using the instruction format suggested in part A. Use the minimal amount of additional hardware and clock cycles/control states. Remember when adding new instructions, don't break the operation of the standard ones. (4 marks)

Solution:

The basic working of the processor remains unchanged. An **additional MUX** is added at the input of ReadRegister2. This is done because there is no explicit reference to source register 2 here, rather the destination register serves as both a rd and rs2. The MUX controls whether to pass rd (in this case) or rs2 in other cases. An additional control line called **Read2Src** is added which controls this MUX. An **additional ALU** which functions as an adder is added which takes the value from rs2 and the data loaded from the memory and adds them. This output is then sent to the MemtoReg MUX. This MUX is also modified from a **2 input MUX to a 3 input MUX** where the output from the adder is given as the 2nd input. The bits of the signal MemtoReg are also changed to 2 now. The values of the control lines would be as follows:

- Branch = 0
- MemRead = 1
- MemtoReg = 10
- ALUOP = 00
- MemWrite = 0
- ALUSrc = 1
- RegWrite = 1
- Read2Src = 1



3. Discuss the effect of the modification in part (B) on the latency of single-cycle non-pipelined CPU. (2 marks)

Solution:

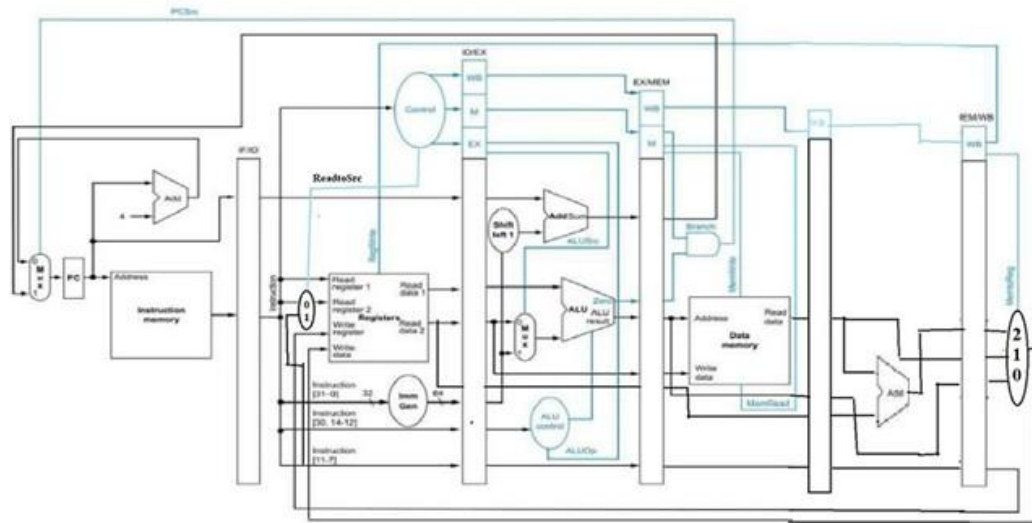
Because of the addition of an extra Adder, the time needed for an instruction to completely execute would now increase.

(its execution will only start after MEM stage, and before WB - therefore additional time)

4. Discuss if the suggested modification in part (B) should be handled by increasing/decreasing the number of pipelining stages that were discussed in class. Draw a pipelined version of the new processor similar to Figure 4.49 of the book. (4 marks)

Solution:

An extra stage can be added for the second ALU. The new MUX is added in the IF stage.



5. Discuss if any new types of data hazards are introduced due to the new instruction? If yes, can they be mitigated through forwarding? Use a multi-cycle pipeline diagram like Figure 4.51 of the book to illustrate the new forwarding paths. (3 marks)

Solution:

Four new kinds of data hazards are created due to the introduction of a new pipeline register, and new adder which consumes register value. Four forwarding paths will be needed from the new pipeline register to the four points in our pipeline which consume register data. The following code sequence illustrates the data hazards.

If we follow the following instructions:

```
addm x9, 0(x28)
```

```
addm x9, 0(x29)
```

```
sd x9, 0(x30)
```

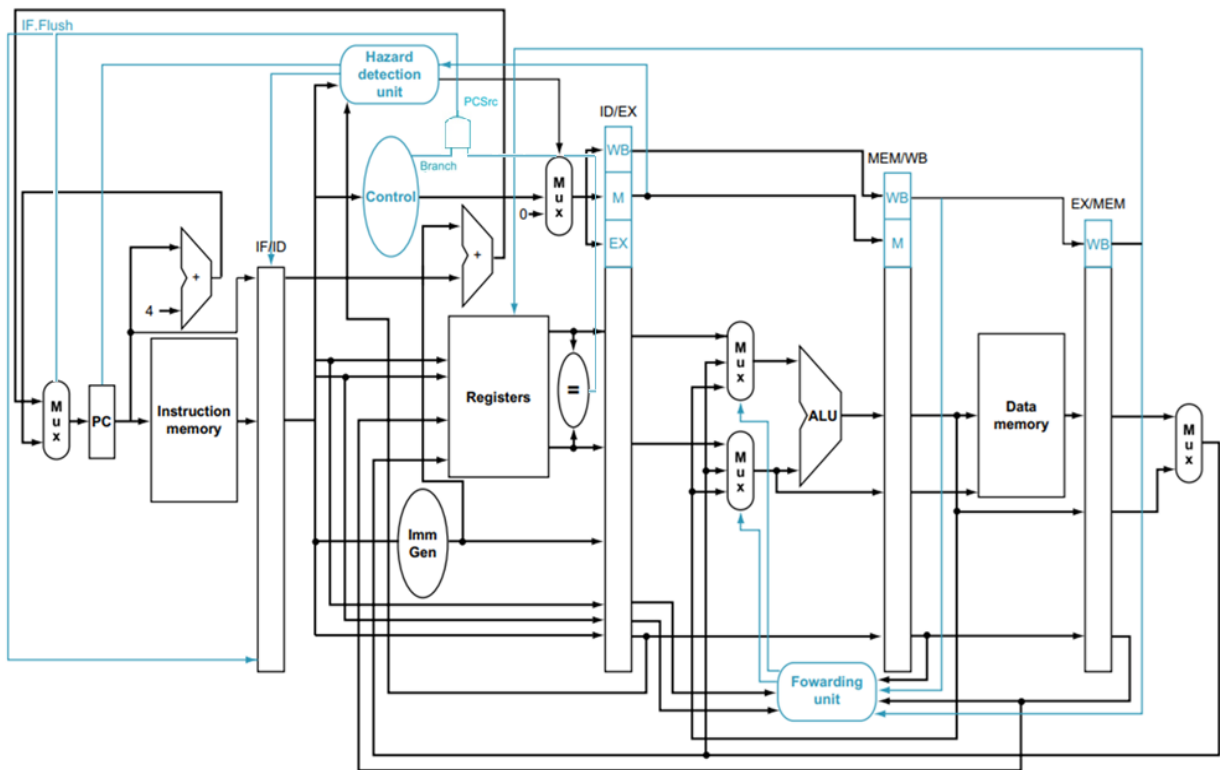
```
add x31, x9, x9
```

Following figure illustrates the forwarding paths needed

Question # 7 (Points: 10)

The pipelined processor's performance might be better if branches take place during the Decode stage rather than the Execute stage. Show how to modify the pipelined processor to move the branch logic to the Decode stage. How do the stall, flush, and forwarding signals change? You can use a printed version of the figures in the book over which you can draw your suggested modifications.

Solution:



Question # 8 (Points: 15)

In a pipelined CPU, the multiply instruction (mul) requires two cycles in the EX stage (EXa and EXb).

1. Explain why this situation causes a structural hazard in the pipeline. Use a pipeline timing chart for a sequence containing two consecutive mul instructions to illustrate the conflict. (Assume there is no stall unit.) (6 marks)

Solution:

Given code sequence:

```
mul x1, x2, x3
mul x5, x4, x6
add x8, x2, x1
```

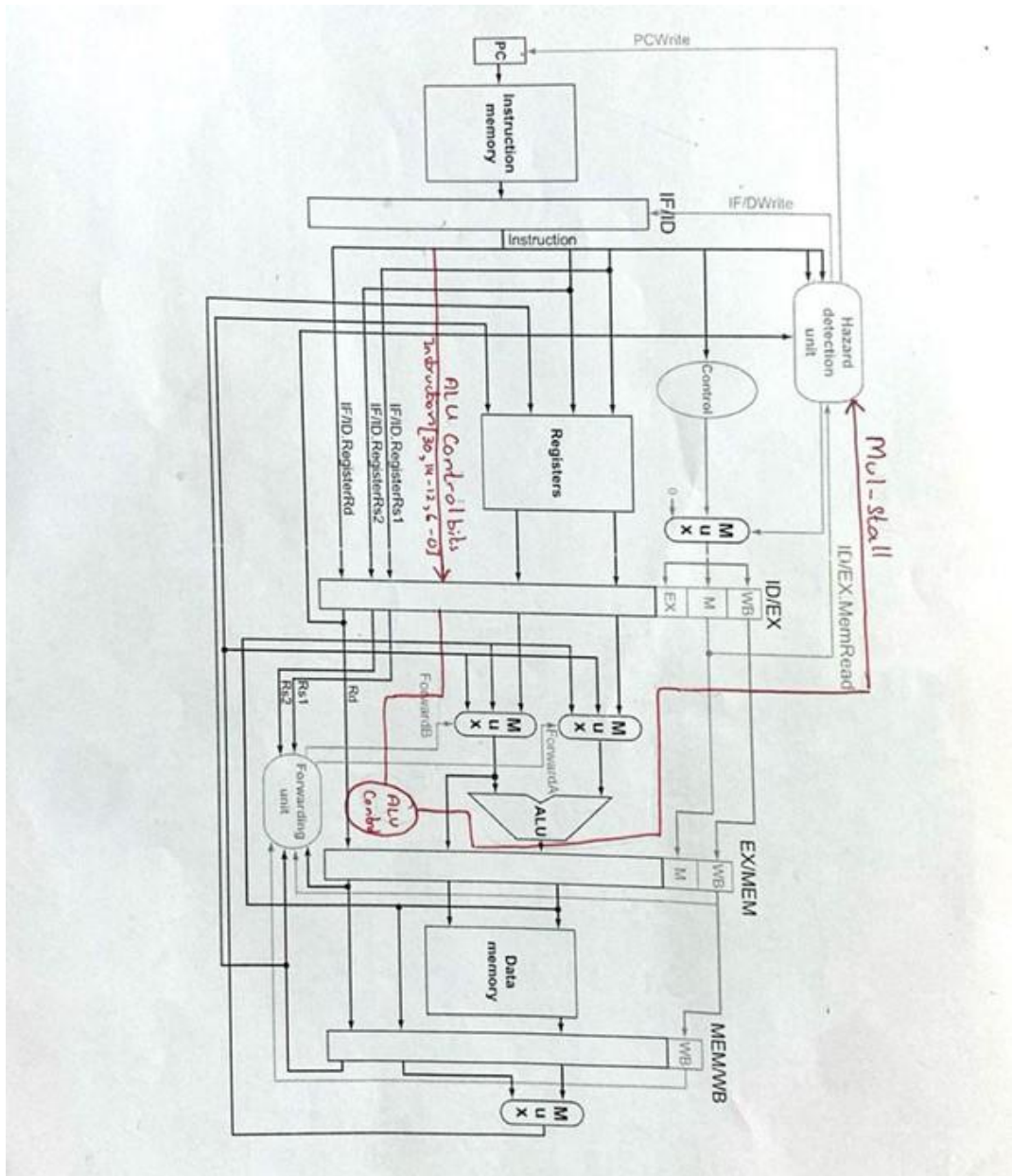
Pipeline timing chart:

Cycle	1	2	3	4	5	6	7
mul x1,x2,x3	IF	ID	EXa	EXb	M	WB	
mul x5,x4,x6		IF	ID	EXa	EXb	M	WB
add x8,x2,x1			IF	ID	EXa	M	WB

Observation: A structural hazard occurs at cycle 4, where both `mul` instructions need the EX unit simultaneously (`EXa` of the second overlaps with `EXb` of the first). In cycle 5, the second `mul` and the `add` instruction also overlap in the EX stage, and their `M` and `WB` stages align, further showing resource contention.

2. Clearly highlight or annotate the modifications needed to support a two-cycle mul instruction using Figure 4.58 from the textbook. Indicate any new control signals, hardware components, or changes to existing control logic that prevent this hazard. (6 marks)

Solution:



To prevent the structural hazard caused by consecutive mul instructions, a new signal MulStall (or EX busy) is added from the EX stage to the Hazard Detection Unit (HDU) in the ID stage. The modified control flow is shown conceptually in Figure 4.58 of the textbook.

- New Signal:
 - MulStall is asserted by the ALU Control in the EX stage whenever a multiply instruction occupies both EXa and EXb.
 - This signal is sent to the HDU to indicate that the EX hardware is busy.
- Hazard Detection Logic Update:

The HDU now stalls the pipeline whenever the MulStall signal is active or a load-use hazard is detected. The updated stall condition is:

$$\text{stall} = (\text{ID/EX.MemRead} \wedge ((\text{ID/EX.RegisterRd} = \text{IF/ID.RegisterRs1}) \vee (\text{ID/EX.RegisterRd} = \text{IF/ID.Register2}))) \vee \text{MulStall}$$

When stall = 1, the HDU performs the following actions:

- PCWrite $\leftarrow$ 0 (freeze Program Counter)
- IF/ID.Write $\leftarrow$ 0 (hold current instruction in Decode)
- ID/EX.control $\leftarrow$ 0 (insert a NOP bubble into the pipeline)

Effect on Pipeline: The next instruction following a mul is stalled in the ID stage until the multiplier finishes execution (i.e., EX busy deasserts). This prevents both EXa and EXb from being occupied simultaneously.

Summary: Adding one feedback signal (MulStall) and slightly extending the hazard detection logic ensures that the pipeline stalls automatically whenever a multi-cycle mul is in progress. No additional forwarding paths or ALU hardware are required

3. Briefly describe how your proposed modifications resolve the hazard. (3 marks)

Solution:

Resolution of the Hazard:

Instruction	1	2	3	4	5	6	7	8	9
mul x1, x2, x3	IF	ID	EXa	EXb	M	WB			
mul becomes nop		IF	ID	☆	☆	☆	☆	☆	
mul x5, x4, x6			IF	ID	EXa	EXb	M	WB	
add becomes nop				IF	ID	☆	☆	☆	☆
add x8, x2, x1					IF	ID	EXa	M	WB

Explanation: In this modified pipeline, the MulStall signal generated by the ALU Control unit activates the hazard detection unit whenever a mul instruction is in progress (occupying both EXa and EXb). The HDU stalls the subsequent instruction in the Decode (ID) stage, as shown by the stall bubbles (☆). This ensures that no new instruction enters the EX stage until the first mul finishes execution.